

CEG 7360 — EMBEDDED SYSTEMS

Laboratory Handout

Laboratory	ESP32 GPIO Programming and Bit Manipulation
Reading	Course text, Chapters 1 and 2
Submit	Project link and report, via Pilot

Purpose: Programming the general-purpose input/output pins of an ESP32 microcontroller in C, and manipulating the individual bits of a byte. Everything runs in a browser-based simulator, so no hardware is required. The code you write here will later run unchanged on a physical ESP32

1. Background

1.1 What a GPIO pin is

A microcontroller communicates with the world through its pins. A general-purpose input/output pin, universally abbreviated GPIO, is one that your program can control directly: you can configure it as an output and set it to a high or low voltage, or configure it as an input and read the voltage another circuit is applying to it.

The ESP32 has 34 GPIO pins in total, of which 25 are brought out to the headers on a typical development board. Each is identified by a number, written GPIO32, GPIO33 and so on. In this laboratory you will use eight of them as outputs, each driving one LED.

Configuring a pin is not automatic. Before a pin can be used, the program must tell the hardware what the pin is for: whether it is an input or an output, whether an internal pull-up resistor should be connected, and whether it should generate an interrupt. In ESP-IDF this is done by filling in a configuration structure and passing it to `gpio_config`. Once configured, `gpio_set_level` drives an output pin high or low.

1.2 Why an embedded program never ends

On a laptop, when the function `main` returns, the operating system takes back control and cleans up the process. A microcontroller has no operating system waiting in the background to take over. If your program finishes, the processor has nothing left to do, and the pins stay in whatever state you left them.

Every embedded program therefore has the same shape: set things up once, then enter a loop that never exits.

```
void app_main(void)
{
    /* initialisation, executed once */

    while (1) {
        /* the program runs here forever */
    }
}
```

Why `app_main` and not `main`? There is a `main`, but it belongs to ESP-IDF. It initialises the hardware, starts the FreeRTOS scheduler, and creates a task which then calls `app_main`. From your point of view, `app_main` is where your program begins.

1.3 Choosing the right integer type

The width of the C type `int` is not fixed by the language. It is 32 bits on the ESP32 and 16 bits on many smaller microcontrollers. Code that assumes a particular width is not portable, and on a small device an oversized variable wastes both memory and time.

Embedded C therefore uses the exact-width types declared in `stdint.h`. Use the narrowest type that holds the range you need, and prefer unsigned unless negative values are genuinely required.

Table 1.1 — Common fixed-width types

Type	Bits	Range	Typical use
<code>uint8_t</code>	8	0 to 255	A byte, a port, a small counter
<code>int8_t</code>	8	−128 to 127	A small signed offset
<code>uint16_t</code>	16	0 to 65,535	A millisecond counter
<code>uint32_t</code>	32	0 to about 4.29 billion	A microsecond timestamp

The output port in this laboratory is exactly eight bits wide, so `uint8_t` is not merely large enough. It is the correct type, because it says what the variable means.

1.4 Hexadecimal

One hexadecimal digit encodes exactly four binary digits, so a byte is exactly two hexadecimal digits and the conversion needs no arithmetic.

<code>0xA5</code>	=	<code>1010 0101</code>
		A 5
bit:		7 6 5 4 3 2 1 0
value:		1 0 1 0 0 1 0 1
weight:		128 64 32 16 8 4 2 1
		128 + 32 + 4 + 1 = 165

The same number in decimal is 165, which tells you nothing about which pins are high. This is why register values in datasheets are always given in hexadecimal, and why you should learn to read it without converting.

1.5 The bitwise operators

These operate on each bit position independently. They are the core tool of this laboratory and of embedded programming generally.

Table 1.2 — Operators

Operator	Name	Effect on one bit position
<code>&</code>	AND	Result is 1 only if both bits are 1. AND with 0 forces a bit to 0; AND with 1 leaves it alone.
<code> </code>	OR	Result is 1 if either bit is 1. OR with 1 forces a bit to 1; OR with 0 leaves it alone.
<code>^</code>	XOR	Result is 1 if the bits differ. XOR with 1 inverts a bit; XOR with 0 leaves it alone.
<code>~</code>	NOT	Inverts every bit.
<code><<</code>	Shift left	Moves bits towards the most significant end. Bits shifted off the top are lost; zeros enter at the bottom.
<code>>></code>	Shift right	Moves bits towards the least significant end.

From these, the four standard operations follow. Learn these four; everything else in this laboratory is built from them.

```

set   bit k :   v = v | (1u << k);
clear bit k :   v = v & ~(1u << k);
toggle bit k :  v = v ^ (1u << k);
test  bit k :   if ((v >> k) & 1u) { ... }

```

Why the pattern works. The expression `1u << k` produces a value with a single 1 in position `k` and zeros elsewhere. That value is called a mask. Because AND, OR and XOR each have a value that leaves a bit unchanged, applying the mask affects only position `k` and leaves the other seven untouched. This is the whole idea.

1.6 Overflow

A `uint8_t` holds values from 0 to 255. Adding one to 255 gives zero. The C standard defines this for unsigned types: the result is reduced modulo 256. You may rely on it.

The corresponding operation on a signed type is different. Adding one to an `int8_t` holding 127 is undefined behaviour. A particular compiler will produce some result, often `-128`, but the standard guarantees nothing, and the behaviour can change with optimisation settings. You will test this in Experiment 2.

1.7 The LED circuit

An LED is a diode. It conducts in one direction only, and it does not limit its own current, so a series resistor is required. Without one, the pin would attempt to supply far more current than it is designed for.

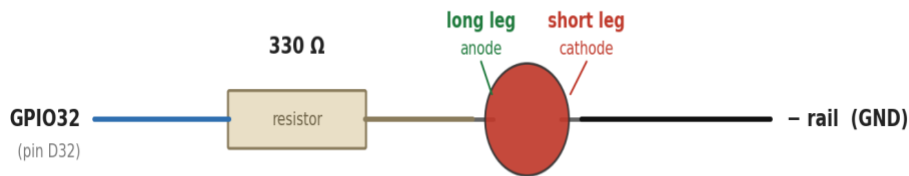


Figure 1 — One channel. The circuit has eight of these.

$$\begin{aligned}
 I &= (V_{\text{supply}} - V_{\text{forward}}) / R \\
 &= (3.3 \text{ V} - 2.0 \text{ V}) / 330 \text{ ohm} \\
 &= 3.9 \text{ mA}
 \end{aligned}$$

The ESP32 datasheet gives 40 mA as the absolute maximum per pin, so 3.9 mA is a comfortable margin.

Table 1.3 — Pin assignment used throughout this course

LED	Bit	GPIO	Board marking	Bit weight
D0	0	GPIO32	D32	1
D1	1	GPIO33	D33	2
D2	2	GPIO25	D25	4
D3	3	GPIO26	D26	8
D4	4	GPIO27	D27	16
D5	5	GPIO14	D14	32
D6	6	GPIO13	D13	64
D7	7	GPIO23	D23	128

Pins that must never be used. GPIO6 to GPIO11 are wired to the flash memory chip; connecting anything to them stops the board running. GPIO0, 2, 5, 12 and 15 control how the chip boots. GPIO34 and above are input-only and cannot drive an LED. Every pin in Table 1.3 is safe.

2. Building the Circuit

You will build this circuit yourself by dragging components onto the canvas and drawing the wires. Wokwi writes the `diagram.json` file for you as you work — you never edit it by hand.

2.1 Create the project

1. Go to wokwi.com and sign in. A free account is sufficient.
2. Go to wokwi.com/projects/new/esp-idf-esp32 — use this address, not the New Project button, so that you get an ESP-IDF project in C.
3. Confirm the editor shows a file named `main.c` containing a function named `app_main`. If you see a file ending in `.ino` with `setup` and `loop`, you have an Arduino project. Delete it and use the address above again.
4. Using the file list, add two files named `pes_hw.h` and `pes_hw.c`. Copy their contents from Appendix A.
5. Rename the project to `PES-Lab1-Surname` and save it.

2.2 Learn the four actions you need

Everything in this section uses only these. Try each one once before you start building.

To do this	Do this
Add a component	Click the blue + button above the canvas, type the part name in the search box, click the result.
Move a component	Click and drag it.
Draw a wire	Click directly on a pin — the small square at the edge of a component. Move the mouse; a wire follows. Click on the destination pin to finish.
Delete something	Click it to select, then press Delete .

Two things worth knowing before you get stuck:

- While drawing a wire, click on empty canvas to place a **corner**. This is how you route wires around components instead of straight through them.
- Press **Escape** to abandon a wire you are part-way through drawing.
- Hover over any pin and Wokwi displays its name. **Use this to confirm you have the right pin before every connection.**

2.3 Build one channel and test it

Do not build all eight yet. Build one, prove it works, then repeat.

1. Click **+**, search for `LED`, and add one. Drag it to the upper-left area of the canvas, leaving room for seven more to its right.
2. Click the LED to select it. A small **settings** control appears — open it and set **color** to `red` and **label** to `D0`.
3. Click **+**, search for `resistor`, and add one. Drag it just below the LED.
4. Select the resistor and set its **value** to `330`.
5. Hover over the ESP32 pins on the right-hand side until you find **D32**. Click it.
6. Click either end of the resistor. A wire now joins GPIO32 to the resistor.
7. Click the resistor's other end, then click the LED's **anode** — hover to confirm it reads **A** before clicking.
8. Click the LED's **cathode (C)**, then click any pin marked **GND** on the ESP32.

Checkpoint. Enter the program from Listing 3.1 and press play. D0 should blink. If it does not, fix it now — the remaining seven channels repeat this exact pattern, and a mistake here becomes eight mistakes.

2.4 Build the remaining seven

Work in batches rather than completing one channel at a time. It is faster and produces a tidier layout.

1. Add seven more LEDs. Drag them into a straight row to the right of D0, evenly spaced.
2. Set each one's colour and label: **D1** and **D2** red, **D3** and **D4** yellow, **D5** to **D7** green.
3. Add seven more resistors, one below each LED, and set every value to `330`.
4. Wire each GPIO to its resistor, using Table 1.3. Take them in order — D33, D25, D26, D27, D14, D13, D23 — and confirm each pin name by hovering before you click.
5. Wire each resistor to its LED anode.

For the grounds, do not run eight separate wires to the ESP32. Instead, connect D0's cathode to D1's cathode, D1's to D2's, and so on along the row, then run a single wire from D7's cathode down to a **GND** pin. All eight cathodes are now joined, and the result is far easier to read.

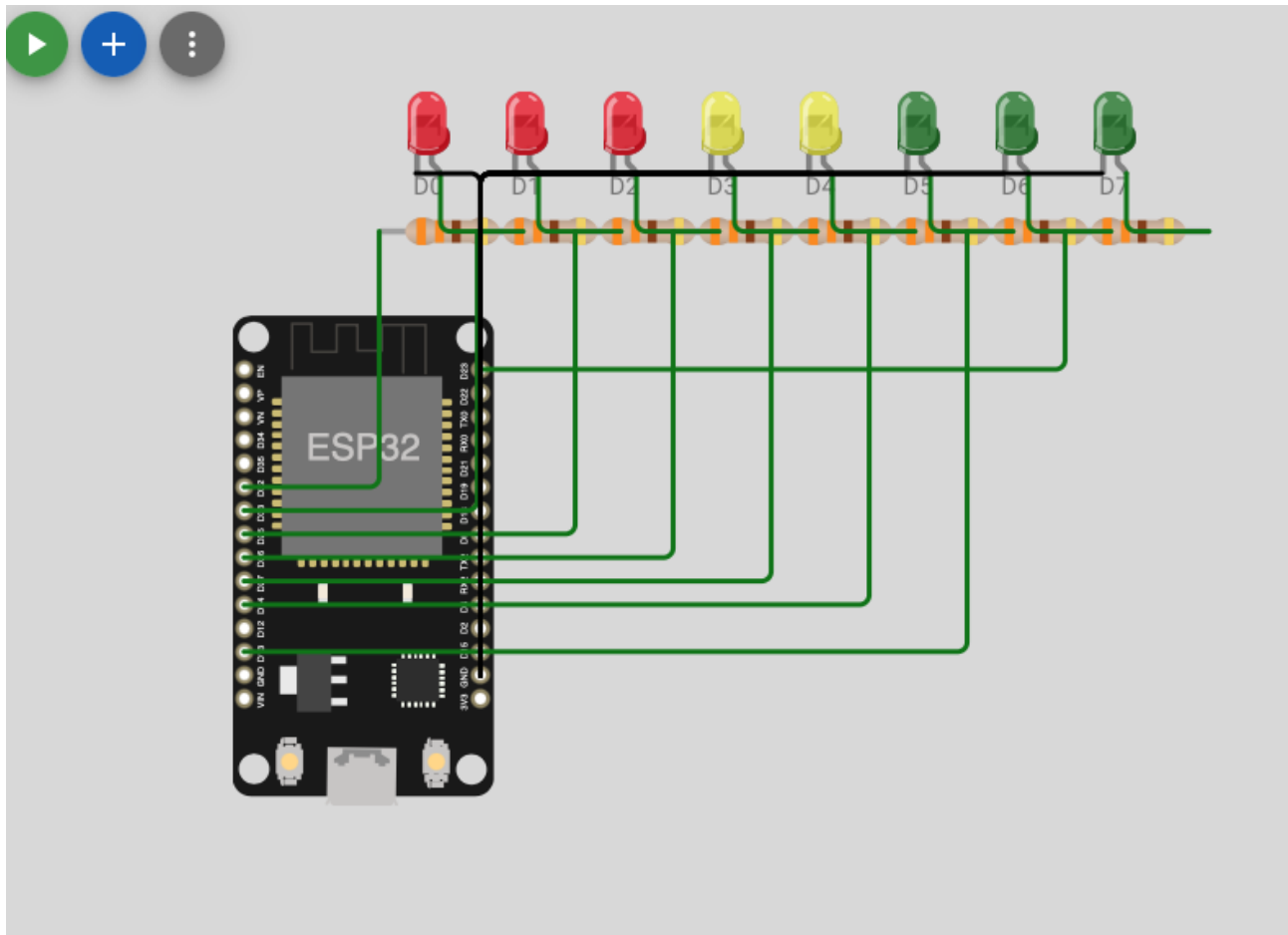
This is what a **common ground** means in a real circuit: one shared return path rather than eight parallel ones.

2.5 Check your work

1. Click the **diagram.json** tab. You did not write this file — Wokwi generated it from what you drew. Read it and find the entry for D0. You should be able to match every line to something you did.
2. Confirm you have exactly **8 LEDs**, **8 resistors**, and **24 wires** (8 GPIO-to-resistor, 8 resistor-to-anode, 7 cathode-to-cathode, 1 cathode-to-GND).
3. Run the program from Listing 4.1. All eight LEDs should now be under program control.

If an LED never lights, the most likely cause is a wire that ends near a pin rather than on it. Click the suspect wire to select it, delete it, and redraw it — hovering to confirm the pin name before each click.

Appendix B contains the finished circuit description. It is there so you can compare against your own work if a channel refuses to behave. Do not paste it in as a shortcut; building the circuit is the exercise



3. Experiment 1 — Controlling One GPIO Pin

Aim

To configure a single pin as an output and drive it high and low at a fixed rate.

Steps

1. Replace the contents of `main.c` with Listing 3.1.
2. Press the play button above the circuit.
3. Observe D0.

Listing 3.1

```

#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "pes_hw.h"

void app_main(void)
{
    pes_gpio_init();          /* configure the eight pins as outputs */

    while (1) {
        led_set(0, true);      /* D0 on */
        vTaskDelay(pdMS_TO_TICKS(500)); /* wait 500 ms */
        led_set(0, false);     /* D0 off */
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

```

Expected result

D0 turns on for half a second, off for half a second, and repeats. The period is one second.

Now break it

1. Delete the while (1) { and its closing brace, leaving the four statements inside.
2. Run it again and describe what happens to D0.
3. Record your answer in Table 8.1, then restore the loop.

4. Experiment 2 — Eight Pins as One Port

Aim

To treat eight separate GPIO pins as a single eight-bit value, and to observe the relationship between that value and the pattern of lit LEDs.

Steps

1. Before running anything, write your predictions in Table 8.2. This is graded, and predictions written afterwards earn nothing.
2. Replace main.c with Listing 4.1 and run it.
3. Watch for at least one minute and complete the observed column of Table 8.2.

Listing 4.1

```

#include <stdio.h>
#include <stdint.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "pes_hw.h"

void app_main(void)
{
    pes_gpio_init();

    uint8_t count = 0;          /* eight bits: exactly the port width */

    while (1) {
        led_write(count);      /* bit 0 drives D0, bit 7 drives D7 */
        printf("dec=%3u  hex=0x%02X\n", count, count);
        count++;
        vTaskDelay(pdMS_TO_TICKS(250));
    }
}

```



```
}
```

Expected result

D0 changes state every 250 ms, D1 every 500 ms, D2 every second, and so on, each LED changing at half the rate of the one to its left. One complete cycle of 256 values takes 64 seconds.

Then try this

1. Change `uint8_t` count to `int8_t` count.
2. Run it and watch what happens as the count approaches 127.
3. Record the result in Table 8.2. Restore `uint8_t` afterwards.

5. Experiment 3 — Modifying Individual Bits

Aim

To write functions that change one bit of a byte and leave the other seven bits unchanged.

In Experiment 2, the function `led_write` replaced all eight LED states at once. This experiment provides finer control. For example, it allows D3 to be switched on without knowing the state of the other seven LEDs.

5.1 The method

A mask is a byte in which the bit positions of interest contain 1 and all other positions contain 0. The expression `1u << k` produces a mask with a single 1 at position `k`.

```
1u << 0 = 0000 0001
1u << 3 = 0000 1000
1u << 7 = 1000 0000
```

The mask is then combined with the value using one of the bitwise operators. Because each operator has an operand value that leaves a bit unchanged, only the selected position is modified.

Table 5.1 — Selecting the operator

Required result at position <code>k</code>	Operator	Mask to use
Set the bit to 1	OR <code> </code>	<code>1u << k</code>
Clear the bit to 0	AND <code>&</code>	<code>~(1u << k)</code>
Invert the bit	XOR <code>^</code>	<code>1u << k</code>

5.2 Worked example — `bit_set`

Task: return `v` with bit `k` set to 1, leaving the other bits unchanged.

1. Build the mask. A single 1 at position `k` is written `1u << k`.
2. Select the operator. Table 5.1 shows that OR sets a bit to 1, and OR with 0 leaves a bit unchanged. OR is therefore correct.
3. Combine the value and the mask.

```
uint8_t bit_set(uint8_t v, uint8_t k)
{
    return v | (1u << k);
}
```

Verification by hand, setting bit 3 of 0xA5:

```
v          = 1010 0101
1u << 3     = 0000 1000
v | mask    = 1010 1101    only bit 3 has changed
```

5.3 Exercise 1 — `bit_clear`

Task: return `v` with bit `k` cleared to 0, leaving the other bits unchanged.

Table 5.1 shows that AND clears a bit, and that the mask must be inverted first. The mask `1u << k` contains a 1 at position `k` and 0 elsewhere, which is the opposite of what is required. Applying `~` produces a 0 at position `k` and 1 everywhere else.

```
1u << 3    = 0000 1000
~(1u << 3) = 1111 0111    0 at position 3, 1 elsewhere
```

Complete the function.

```
uint8_t bit_clear(uint8_t v, uint8_t k)
{
    return v & _____;    /* complete this expression */
}
```

Common error. Writing `v & (1u << k)` without the inversion keeps bit `k` and clears all other bits. This is the opposite of the required behaviour. Test with `bit_clear(0xFF, 3)`, which must return `0xF7`.

5.4 Exercise 2 — bit_toggle

Task: return `v` with bit `k` inverted, leaving the other bits unchanged.

This function has the same form as `bit_set`. Only the operator changes. Use Table 5.1 to select it.

```
uint8_t bit_toggle(uint8_t v, uint8_t k)
{
    return v ____ (1u << k);    /* complete the operator */
}
```

5.5 Worked example — low_nibble

A nibble is a group of four bits. A byte contains two nibbles.

Task: return only the lower four bits of `v`, with the upper four bits set to 0.

The mask must contain 1 in the four positions to keep and 0 in the four positions to remove. That mask is `0000 1111`, which is `0x0F`. AND with 1 leaves a bit unchanged, and AND with 0 clears it.

```
uint8_t low_nibble(uint8_t v)
{
    return v & 0x0F;

    v          = 1010 0101
    0x0F       = 0000 1111
    v & 0x0F   = 0000 0101    the lower nibble, 0x05
```

5.6 Exercise 3 — high_nibble

Task: return the upper four bits of `v`, moved down to positions 3 to 0. For example, `high_nibble(0xA5)` must return `0x0A`, not `0xA0`.

This exercise requires two steps rather than one.

```
v          = 1010 0101

Step 1. Move the upper nibble down by four positions.
v >> 4    = 0000 1010

Step 2. Remove anything remaining in the upper positions.
& 0x0F    = 0000 1010    the result, 0x0A

uint8_t high_nibble(uint8_t v)
{
    return (v ____ 4) & 0x0F;    /* which shift operator? */
}
```

Common error. Writing `v & 0xF0` selects the correct bits but leaves them in positions 7 to 4, so the function returns `0xA0`. The bits must be moved down first.

5.7 Exercise 4 — swap_nibbles

Task: exchange the two nibbles of `v`. For example, `0xAB` becomes `0xBA`.

This requires two shifts in opposite directions, combined with OR.

```
v      = 1010 0101   (0xA5)

Move the lower nibble up:
v << 4 = 0101 0000

Move the upper nibble down:
v >> 4 = 0000 1010

Combine the two results with OR:
OR     = 0101 1010   (0x5A)

uint8_t swap_nibbles(uint8_t v)
{
    return (uint8_t)((v << 4) ____ (v >> 4)); /* which operator? */
}
```

The purpose of the cast. In C, a `uint8_t` value is converted to `int` before a shift is applied, so `v << 4` produces a result wider than eight bits. Assigning it to a `uint8_t` discards the upper bits. The cast to `(uint8_t)` states that this is intended and prevents a compiler warning. The cast is provided for you in this exercise.

5.8 Exercise 5 — rotl8

Task: move every bit one position to the left. The bit leaving position 7 must reappear at position 0.

A left shift alone discards the bit in position 7. That bit must be recovered separately and placed in position 0.

```
v      = 1000 0001   (0x81)

Move everything left. Bit 7 is lost:
v << 1 = 0000 0010

Recover the original bit 7 by moving it to position 0:
v >> 7 = 0000 0001

Combine the two results with OR:
OR     = 0000 0011   (0x03)

uint8_t rotl8(uint8_t v)
{
    return (uint8_t)((v << 1) | (v >> ____ )); /* how many positions? */
}
```

5.9 Testing your functions

Copy Listing 5.1 into your program. Run it and confirm that every line reports PASS.

Listing 5.1 — Test program

```
#include <stdio.h>
#include <stdint.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "pes_hw.h"

/* Place your five completed functions here, together with the two
   worked examples bit_set and low_nibble. */

static int fails = 0;
```

```

#define CHECK(expr, want)
do {
    uint8_t got = (uint8_t)(expr);
    printf("%-26s got=0x%02X want=0x%02X  %s\n",
           #expr, got, (uint8_t)(want),
           got == (uint8_t)(want) ? "PASS" : "***FAIL***");
    if (got != (uint8_t)(want)) fails++;
} while (0)

void app_main(void)
{
    pes_gpio_init();

    CHECK(bit_set(0x00, 0), 0x01);
    CHECK(bit_set(0x00, 7), 0x80);
    CHECK(bit_clear(0xFF, 0), 0xFE);
    CHECK(bit_clear(0xFF, 7), 0x7F);
    CHECK(bit_clear(0xFF, 3), 0xF7);
    CHECK(bit_toggle(0xAA, 0), 0xAB);
    CHECK(bit_toggle(0xAA, 1), 0xA8);
    CHECK(low_nibble(0xA5), 0x05);
    CHECK(low_nibble(0xFF), 0x0F);
    CHECK(high_nibble(0xA5), 0x0A);
    CHECK(high_nibble(0xFF), 0x0F);
    CHECK(swap_nibbles(0xA5), 0x5A);
    CHECK(swap_nibbles(0xFF), 0xFF);
    CHECK(rotl8(0x01), 0x02);
    CHECK(rotl8(0x80), 0x01);

    printf("\n--- %d failure(s) ---\n", fails);

    /* One LED moves left and returns to D0. */
    uint8_t v = 0x01;
    while (1) {
        led_write(v);
        v = rotl8(v);
        vTaskDelay(pdMS_TO_TICKS(150));
    }
}

```

Expected result: 15 PASS lines and zero failures. One LED then moves one position left on each step and returns from D7 to D0.

If the LED disappears instead of returning to D0. Your rotl8 is performing a plain shift, so the bit in position 7 is being discarded. Check the second shift amount in Exercise 5.

5.11 Complete assembled code

```

#include <stdio.h>
#include <stdint.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "pes_hw.h"

/* ----- the eight bit functions ----- */

uint8_t bit_set(uint8_t v, uint8_t k)

```

```

{
    return v | (1u << k);
}

uint8_t bit_clear(uint8_t v, uint8_t k)
{
    return v & ~(1u << k);
}

uint8_t bit_toggle(uint8_t v, uint8_t k)
{
    return v ^ (1u << k);
}

uint8_t low_nibble(uint8_t v)
{
    return v & 0x0F;
}

uint8_t high_nibble(uint8_t v)
{
    return (v >> 4) & 0x0F;
}

uint8_t swap_nibbles(uint8_t v)
{
    return (uint8_t)((v << 4) | (v >> 4));
}

uint8_t rotl8(uint8_t v)
{
    return (uint8_t)((v << 1) | (v >> 7));
}

/* ----- test harness ----- */

static int fails = 0;

#define CHECK(expr, want) \
do { \
    uint8_t got = (uint8_t)(expr); \
    printf("%-26s got=0x%02X want=0x%02X %s\n", \
        #expr, got, (uint8_t)(want), \
        got == (uint8_t)(want) ? "PASS" : "***FAIL***"); \
    if (got != (uint8_t)(want)) fails++; \
} while (0)

void app_main(void)
{

```

```

pes_gpio_init();

CHECK(bit_set(0x00, 0), 0x01);
CHECK(bit_set(0x00, 7), 0x80);
CHECK(bit_clear(0xFF, 0), 0xFE);
CHECK(bit_clear(0xFF, 7), 0x7F);
CHECK(bit_clear(0xFF, 3), 0xF7);
CHECK(bit_toggle(0xAA, 0), 0xAB);
CHECK(bit_toggle(0xAA, 1), 0xA8);
CHECK(low_nibble(0xA5), 0x05);
CHECK(low_nibble(0xFF), 0x0F);
CHECK(high_nibble(0xA5), 0x0A);
CHECK(high_nibble(0xFF), 0x0F);
CHECK(swap_nibbles(0xA5), 0x5A);
CHECK(swap_nibbles(0xFF), 0xFF);
CHECK(rotl8(0x01), 0x02);
CHECK(rotl8(0x80), 0x01);

printf("\n--- %d failure(s) ---\n", fails);

/* One LED moves left and returns to D0. */
uint8_t v = 0x01;
while (1) {
    led_write(v);
    v = rotl8(v);
    vTaskDelay(pdMS_TO_TICKS(150));
}
}

```

6. Experiment 4 — Gray Code

Aim

In Experiment 2, several LEDs changed at the same time. For example, the value 3 is 0011 and the value 4 is 0100, so three bits change in one step.

Gray code is an ordering of the same 256 values in which exactly one bit changes between any two consecutive values. In this experiment the conversion function is provided. Your task is to verify that the single-bit property holds.

6.1 The conversion, provided

```
static uint8_t to_gray(uint8_t n)
{
    return (uint8_t)(n ^ (n >> 1));
}
```

The conversion uses one right shift and one XOR operation. The first eight values are shown below.

n	binary	gray	bit that changed
0	0000	0000	
1	0001	0001	bit 0
2	0010	0011	bit 1
3	0011	0010	bit 0
4	0100	0110	bit 2
5	0101	0111	bit 0
6	0110	0101	bit 1
7	0111	0100	bit 0

In the binary column, the step from 3 to 4 changes three bits. In the Gray column, no step changes more than one bit.

6.2 How to verify the property

Applying XOR to two consecutive values produces a 1 in every position where they differ. Counting the bits set in that result gives the number of bits that changed. For a correct sequence this count is always 1.

A function to count set bits is provided below, because it is not required until the homework.

```
static uint8_t popcount8(uint8_t v)
{
    uint8_t n = 0;
    while (v) { v &= (uint8_t)(v - 1); n++; }
    return n;
}
```

6.3 Procedure

Complete the three marked lines in Listing 6.1 and run the program.

Listing 6.1 — Gray code verification

```
#include <stdio.h>
#include <stdint.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "pes_hw.h"

static uint8_t popcount8(uint8_t v)
{
    uint8_t n = 0;
    while (v) { v &= (uint8_t)(v - 1); n++; }
    return n;
}
```



```

}

static uint8_t to_gray(uint8_t n)
{
    return (uint8_t)(n ^ (n >> 1));
}

void app_main(void)
{
    pes_gpio_init();

    int    fails = 0;
    uint8_t prev = to_gray(0);

    for (int n = 0; n < 256; n++) {
        uint8_t g = to_gray((uint8_t)n);

        /* Line 1: display the Gray code value on the LEDs. */
        _____ ;

        if (n > 0) {
            /* Line 2: XOR the current and previous values so that
               each differing bit position becomes 1.          */
            uint8_t changed = _____ ;

            /* Line 3: count how many bits changed. */
            uint8_t howmany = _____ ;

            if (howmany != 1) {
                printf("FAIL at n=%d: %u bits changed\n", n, howmany);
                fails++;
            }
        }
        prev = g;

        printf("n=%3d bin=0x%02X gray=0x%02X\n", n, n, g);
        vTaskDelay(pdMS_TO_TICKS(120));
    }

    printf("\n=== %d failure(s) in 255 transitions ===\n", fails);
    while (1) { vTaskDelay(pdMS_TO_TICKS(1000)); }
}

```

Expected result: zero failures. Only one LED changes at each step, which is visibly different from Experiment 2.

6.4 Question

A rotary position sensor reads several tracks on a disc at the same time, using one detector per track. Explain why such discs are printed in Gray code rather than ordinary binary. Describe what can happen when the disc is read while it is positioned between two values. Record your answer in your report.

7. Experiment 5 — Moving Patterns

Aim

To produce moving patterns on the eight LEDs, and to identify when a function must retain information between calls.

Each pattern function advances the pattern by one step and then returns. No pattern function contains a delay. The main loop calls the function, waits, and calls it again.

7.1 Worked example — a moving LED

The first pattern moves one LED to the left and returns it to D0. It is the rotation from Exercise 5, so no new code is required.

```
static uint8_t walk(uint8_t v)
{
    return rotl8(v);
}
```

This function needs no additional information. The current LED pattern is sufficient to determine the next one.

7.2 Exercise — a bouncing LED

The second pattern moves one LED to the left until it reaches D7, then moves it right until it reaches D0, and repeats.

This pattern cannot be computed from the LED value alone. The value 0x04 could be moving left or moving right, and the function cannot determine which. The direction must therefore be stored in a variable that survives between calls. A variable declared static does this.

Complete the four marked lines.

```
static uint8_t bounce(void)
{
    static uint8_t v = 0x01; /* static: value is kept between calls */
    static int8_t dir = +1; /* +1 means moving left */

    if (dir > 0) { /* currently moving left */
        if (v == 0x80) { /* reached D7 */
            dir = -1; /* change direction */
            v = (uint8_t)(v >> 1); /* first step to the right */
        } else {
            v = (uint8_t)(v ____ 1); /* Line 1: keep moving left */
        }
    } else {
        if (v == _____) { /* currently moving right */
            dir = ____; /* Line 2: which value is D0? */
            v = (uint8_t)(v << 1); /* Line 3: change direction */
        } else {
            v = (uint8_t)(v ____ 1); /* Line 4: keep moving right */
        }
    }
    return v;
}
```

7.3 Running the patterns

```
void app_main(void)
{
    pes_gpio_init();
}
```

```

uint8_t w = 0x01;

while (1) {
    for (int i = 0; i < 24; i++) {
        w = walk(w);
        led_write(w);
        vTaskDelay(pdMS_TO_TICKS(120));
    }
    for (int i = 0; i < 24; i++) {
        led_write(bounce());
        vTaskDelay(pdMS_TO_TICKS(120));
    }
}

```

Expected result: one LED moves left and wraps around for 24 steps, then one LED moves left and right between the two ends for 24 steps. The sequence repeats.

7.4 Extension, optional

If time permits, write two further patterns.

- A bar that fills from D0 upward one LED at a time until all eight are lit, then empties again.
- Two lit LEDs that start at D0 and D7, move toward each other until they meet, then move apart.

11. Results to Record

Table 11.1 — Experiment 1

Observation	Your answer
Measured period of D0	
Behaviour when the while loop is removed	
Explanation of that behaviour	

Table 11.2 — Experiment 2. Complete the prediction column before running the program.

Question	Prediction	Observed
Which LED changes state most frequently?		
At what count does D7 first light, in decimal?		
The same count in hexadecimal		
What value follows 255 in a uint8_t?		
Time for one complete cycle		
Behaviour near 127 with an int8_t		

Table 11.3 — Experiment 3. Record your completed expressions.

Function	Your expression	Tests passed
bit_clear		
bit_toggle		
high_nibble		
swap_nibbles		
rotl8		

12. Submission

- Your Wokwi project link, shared and tested in a private browser window.
- Tables 11.1 to 11.4, completed.
- A screenshot of your test output showing all checks passing.
- Your answers to the questions in Experiments.

Appendix A — Provided Source Files

Copy these two files into your project without modification. They configure the eight pins and allow them to be used as a single byte.

pes_hw.h

```
#ifndef PES_HW_H
#define PES_HW_H

#include <stdint.h>
#include <stdbool.h>
#include "driver/gpio.h"

#define LED_COUNT 8
extern const gpio_num_t LED_PIN[LED_COUNT];

void pes_gpio_init(void);          /* configure all eight as outputs */
void led_write(uint8_t value);    /* bit 0 drives D0 */
uint8_t led_read(void);          /* the last value written */
void led_set(uint8_t bit, bool on);

#endif
```

pes_hw.c

```
#include "pes_hw.h"

const gpio_num_t LED_PIN[LED_COUNT] = {
    GPIO_NUM_32, GPIO_NUM_33, GPIO_NUM_25, GPIO_NUM_26,
    GPIO_NUM_27, GPIO_NUM_14, GPIO_NUM_13, GPIO_NUM_23,
};

/* A copy of the last value written, so that one bit can be changed
   without altering the other seven. */
static uint8_t s_shadow = 0x00;

void pes_gpio_init(void)
{
    uint64_t mask = 0;
    for (int i = 0; i < LED_COUNT; i++) {
        mask |= (1ULL << LED_PIN[i]);
    }

    gpio_config_t cfg = {
        .pin_bit_mask = mask,
        .mode          = GPIO_MODE_OUTPUT,
        .pull_up_en     = GPIO_PULLUP_DISABLE,
        .pull_down_en   = GPIO_PULLEDOWN_DISABLE,
        .intr_type      = GPIO_INTR_DISABLE,
    };
    gpio_config(&cfg);

    led_write(0x00);
}

void led_write(uint8_t value)
{
    s_shadow = value;
    for (int i = 0; i < LED_COUNT; i++) {
        gpio_set_level(LED_PIN[i], (value >> i) & 1u);
    }
}
```

```
uint8_t led_read(void) { return s_shadow; }

void led_set(uint8_t bit, bool on)
{
    if (bit >= LED_COUNT) return;
    led_write(on ? (s_shadow | (1u << bit))
              : (s_shadow & ~(1u << bit)));
}
```

Appendix B — Reference Circuit

Use this only to compare against your own circuit if a channel does not work. Building the circuit is part of the exercise.

```
{
  "version": 1, "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-v1", "id": "esp", "top": 0, "left": 0 },
    { "type": "wokwi-led", "id": "d0", "top": -100, "left": 60, "attrs": { "color": "red",
"label": "D0" } },
    { "type": "wokwi-led", "id": "d1", "top": -100, "left": 100, "attrs": { "color": "red",
"label": "D1" } },
    { "type": "wokwi-led", "id": "d2", "top": -100, "left": 140, "attrs": { "color": "red",
"label": "D2" } },
    { "type": "wokwi-led", "id": "d3", "top": -100, "left": 180, "attrs": { "color": "yellow",
"label": "D3" } },
    { "type": "wokwi-led", "id": "d4", "top": -100, "left": 220, "attrs": { "color": "yellow",
"label": "D4" } },
    { "type": "wokwi-led", "id": "d5", "top": -100, "left": 260, "attrs": { "color": "green",
"label": "D5" } },
    { "type": "wokwi-led", "id": "d6", "top": -100, "left": 300, "attrs": { "color": "green",
"label": "D6" } },
    { "type": "wokwi-led", "id": "d7", "top": -100, "left": 340, "attrs": { "color": "green",
"label": "D7" } },
    { "type": "wokwi-resistor", "id": "r0", "top": -40, "left": 60, "attrs": { "value": "330"
} },
    { "type": "wokwi-resistor", "id": "r1", "top": -40, "left": 100, "attrs": { "value": "330"
} },
    { "type": "wokwi-resistor", "id": "r2", "top": -40, "left": 140, "attrs": { "value": "330"
} },
    { "type": "wokwi-resistor", "id": "r3", "top": -40, "left": 180, "attrs": { "value": "330"
} },
    { "type": "wokwi-resistor", "id": "r4", "top": -40, "left": 220, "attrs": { "value": "330"
} },
    { "type": "wokwi-resistor", "id": "r5", "top": -40, "left": 260, "attrs": { "value": "330"
} },
    { "type": "wokwi-resistor", "id": "r6", "top": -40, "left": 300, "attrs": { "value": "330"
} },
    { "type": "wokwi-resistor", "id": "r7", "top": -40, "left": 340, "attrs": { "value": "330"
} },
  ],
  "connections": [
    [ "esp:D32", "r0:1", "green", [] ], [ "r0:2", "d0:A", "green", [] ],
    [ "esp:D33", "r1:1", "green", [] ], [ "r1:2", "d1:A", "green", [] ],
    [ "esp:D25", "r2:1", "green", [] ], [ "r2:2", "d2:A", "green", [] ],
    [ "esp:D26", "r3:1", "green", [] ], [ "r3:2", "d3:A", "green", [] ],
    [ "esp:D27", "r4:1", "green", [] ], [ "r4:2", "d4:A", "green", [] ],
    [ "esp:D14", "r5:1", "green", [] ], [ "r5:2", "d5:A", "green", [] ],
    [ "esp:D13", "r6:1", "green", [] ], [ "r6:2", "d6:A", "green", [] ],
    [ "esp:D23", "r7:1", "green", [] ], [ "r7:2", "d7:A", "green", [] ],
    [ "d0:C", "d1:C", "black", [] ], [ "d1:C", "d2:C", "black", [] ],
    [ "d2:C", "d3:C", "black", [] ], [ "d3:C", "d4:C", "black", [] ],
    [ "d4:C", "d5:C", "black", [] ], [ "d5:C", "d6:C", "black", [] ],
    [ "d6:C", "d7:C", "black", [] ], [ "d7:C", "esp:GND.1", "black", [] ]
  ]
}
```

Appendix C — Troubleshooting

Symptom	Probable cause
The editor shows setup() and loop()	An Arduino project was created. Create a new project using the address in Section 2.1.
undefined reference to led_write	pes_hw.c was added but is not being compiled with the project.
The simulation does not start	A compilation error. Read the first message reported, not the last.
One LED never lights	A wire refers to the wrong pin, or the anode and cathode are exchanged.
The simulation stops on its own	Free accounts have a run-time limit. Restart the simulation.
The counter appears to skip values	The console cannot print at that rate. The LEDs show the correct sequence.
A warning about conversion altering a value	A shift produced a value wider than eight bits. Add an explicit (uint8_t) cast.